Divide-and-Conquer Technique: Merge Sort, Quick Sort

Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

Merge Sort and Quick Sort

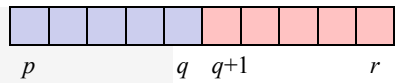
Two well-known sorting algorithms adopt this divide-and-conquer strategy

- Merge sort
  - Divide step is trivial – just split the list into two equal parts
  - Work is carried out in the conquer step by merging two sorted lists
- Quick sort
  - Work is carried out in the divide step using a pivot element
  - Conquer step is trivial

Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

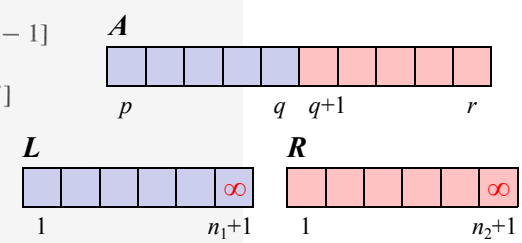
Merge Sort: Algorithm

```
MERGE-SORT(A, p, r)
if p < r
    then q ← ⌊(p + r)/2⌋
         MERGE-SORT(A, p, q)
         MERGE-SORT(A, q + 1, r)
         MERGE(A, p, q, r)
```



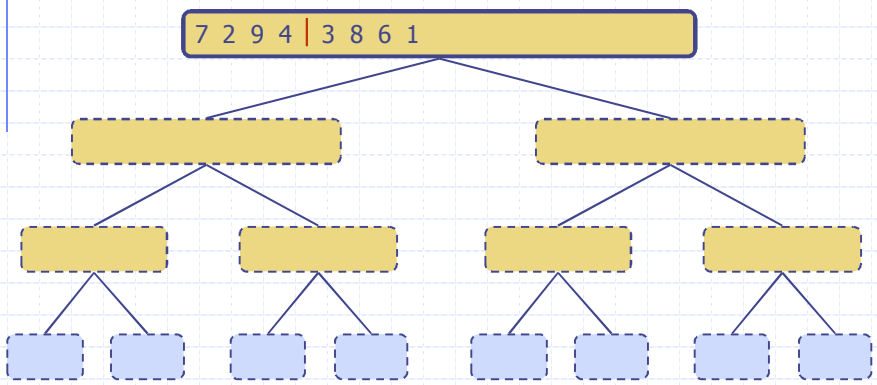
Merge Sort: Algorithm

```
MERGE(A, p, q, r)
n1 ← q − p + 1
n2 ← r − q
create arrays L[1 . . n1 + 1] and R[1 . . n2 + 1]
for i ← 1 to n1
    do L[i] ← A[p + i − 1]
for j ← 1 to n2
    do R[j] ← A[q + j]
L[n1 + 1] ← ∞
R[n2 + 1] ← ∞
i ← 1
j ← 1
for k ← p to r
    do if L[i] ≤ R[j]
          then A[k] ← L[i]
               i ← i + 1
          else A[k] ← R[j]
               j ← j + 1
```



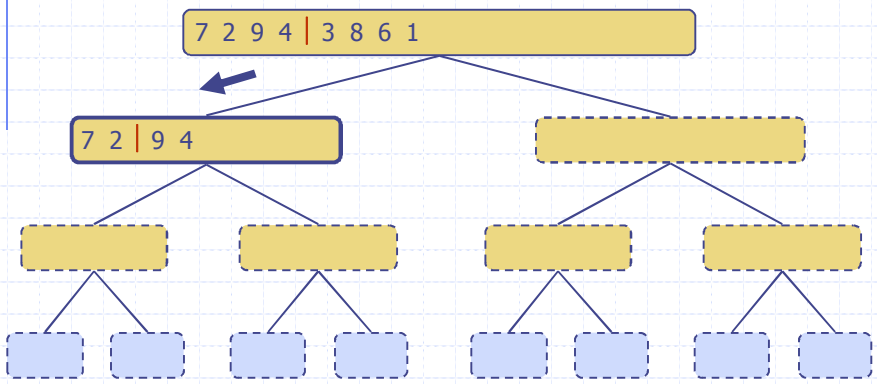
Merge Sort: Example

- Divide



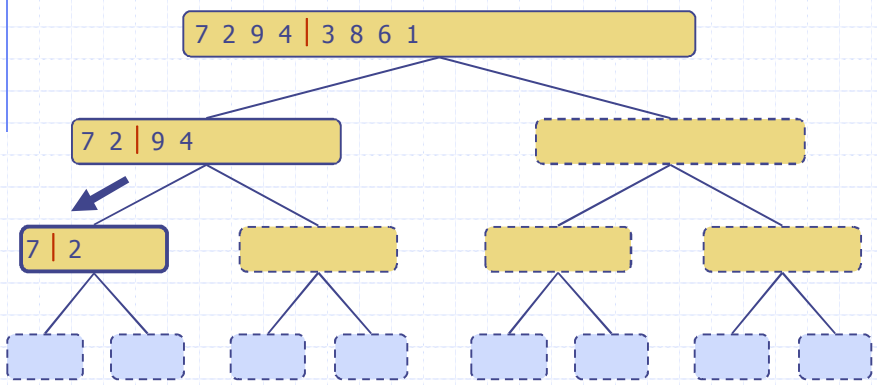
Merge Sort: Example

- Recursive call, divide



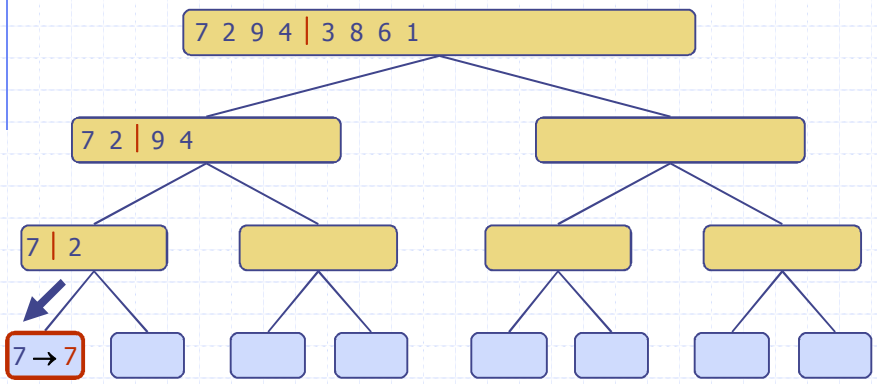
Merge Sort: Example

- Recursive call, partition



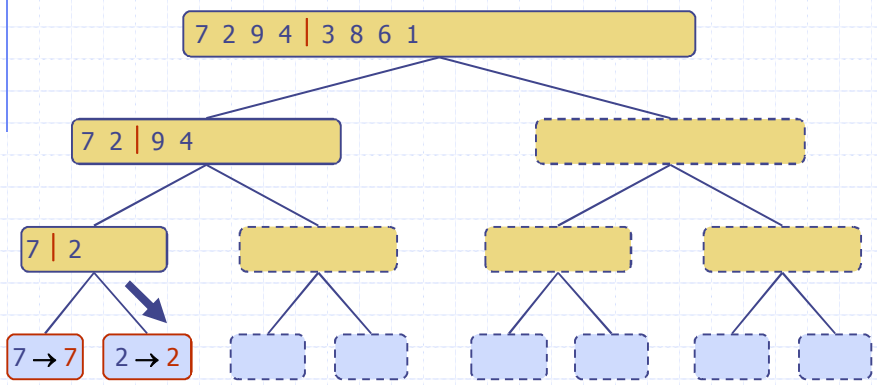
Merge Sort: Example

- Recursive call, base case



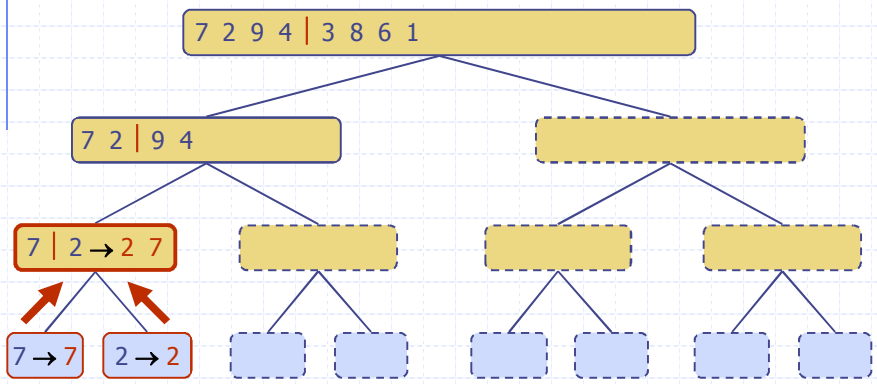
Merge Sort: Example

- Recursive call, base case



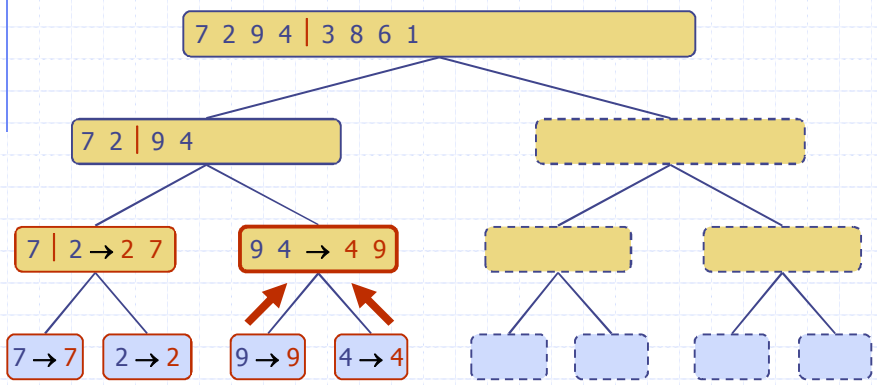
Merge Sort: Example

- Merge



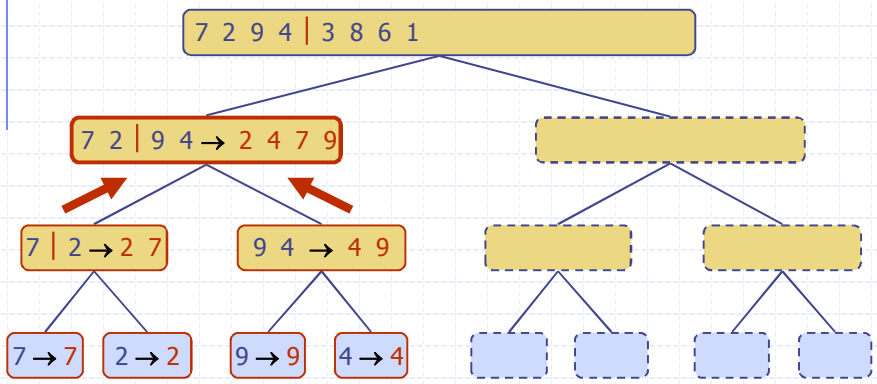
Merge Sort: Example

- Recursive call, …, base case, merge



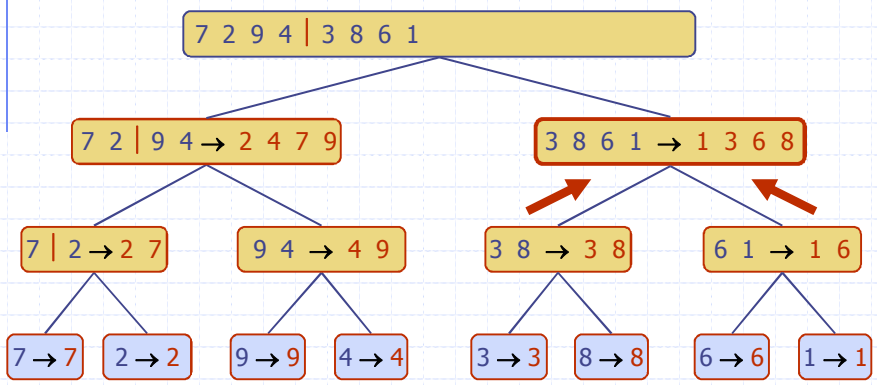
Merge Sort: Example

- Merge



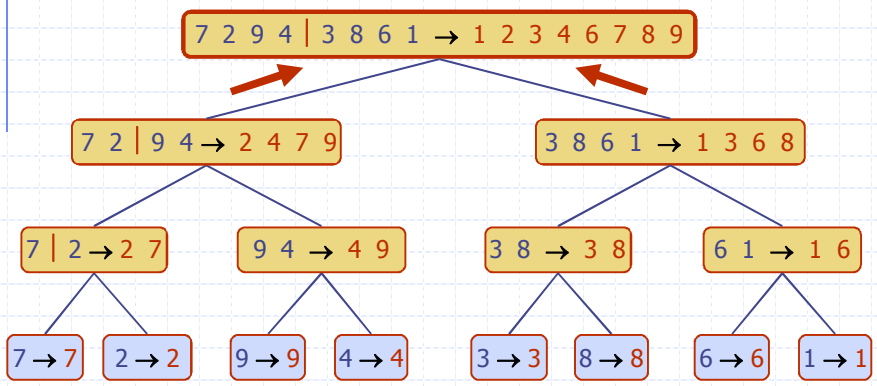
Merge Sort: Example

- Recursive call, …, merge, merge



Merge Sort: Example

- Merge



Merge Sort: Running Time

The recurrence for the worst-case running time T(n) is

$$\mathrm{T}(n) \le \begin{cases} O(1) & \text{if } n = 1 \\ \underbrace{T(\lceil n/2 \rceil)}_{\text{solve left half}} + \underbrace{T(\lfloor n/2 \rfloor)}_{\text{solve right half}} + \underbrace{O(1)}_{\text{dividing}} + \underbrace{O(n)}_{\text{merging}} & \text{otherwise} \end{cases}$$

equivalently

$$\mathrm{T}(n) = \begin{cases} O(1) & \text{if } n = 1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{O(n)}_{\text{dividing + merging}} & \text{otherwise} \end{cases}$$

Merge Sort: Running Time

The recurrence for the worst-case running time T(n) is

$$\mathrm{T}(n) = \begin{cases} O(1) & \text{if } n = 1 \\ 2\mathrm{T}(n/2) + O(n) & \text{if } n > 1 \end{cases}$$

equivalently

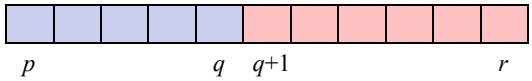
$$\mathrm{T}(n) = \begin{cases} b & \text{if } n = 1 \\ 2\mathrm{T}(n/2) + bn & \text{if } n > 1 \end{cases}$$

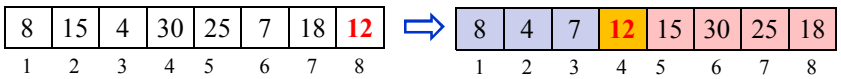
By solving the recurrence, we get

$$T(n) = O(n \log n)$$

Quick Sort: Algorithm

- Another divide-and-conquer algorithm
  - The array A[*p*..*r*] is *partitioned* into two non-empty subarrays A[*p*..*q*] and A[*q*+1..*r*]



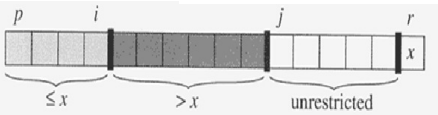


    - Invariant: All elements in A[*p*..*q*] are less than all elements in A[*q*+1..*r*]
  - The subarrays are recursively sorted by calls to quicksort
  - Unlike merge sort, no combining step: two subarrays form an already-sorted array

Quick Sort: Algorithm

```
QUICKSORT(A, p, r)
if p < r
   then q ← PARTITION(A, p, r)
        QUICKSORT(A, p, q − 1)
        QUICKSORT(A, q + 1, r)
```

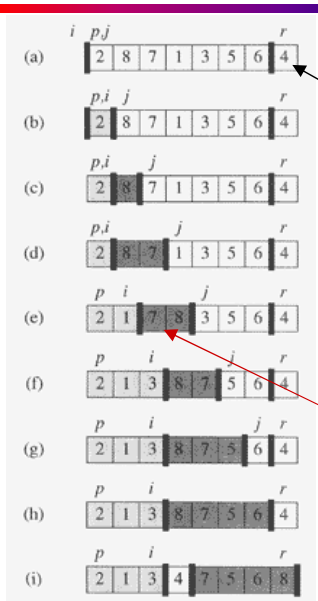
```
PARTITION(A, p, r)
x ← A[r]
i ← p − 1
for j ← p to r − 1
    do if A[j] ≤ x
          then i ← i + 1
               exchange A[i] ↔ A[j]
exchange A[i + 1] ↔ A[r]
return i + 1
```



Quick Sort: Algorithm (Partition)

- Clearly, all the actions take place in the **`partition()`** function
  - Rearranges the subarrays in place
  - End result:
    - Two subarrays
    - All values in first subarray ≤ all values in the second
  - Returns the index of the "pivot" element separating the two subarrays

Quick Sort: Algorithm



pivot

From i +1 to j is a window of elements > A[r]. The cursor j moves right one step at a time.

If the cursor j "discovers" an element ≤ A[r], then this element is swapped with the front element of the window, effectively moving the window right one step; if it discovers an element > A[r], then the window simply becomes longer one unit.

Quick Sort: Analysis

- *What will be the worst case for the algorithm?*
  - Partition is always unbalanced
- *What will be the best case for the algorithm?*
  - Partition is perfectly balanced
- *Which is more likely?*
  - The is almost balanced …
- *Will any particular input elicit the worst case?*
  - Yes: Already-sorted input

Quick Sort: Worst-case Running Time

The recurrence for the worst-case running time T(n) is
[Partition is always unbalanced]

$$\mathrm{T}(n) \le \begin{cases} O(1) & \text{if } n = 1 \\ \underbrace{T(1)}_{\text{solve for single element}} + \underbrace{T(n-1)}_{\text{solve for n-1 element}} + \underbrace{O(n)}_{\text{dividing}} + \underbrace{0}_{\text{merging}} & \text{otherwise} \end{cases}$$

equivalently

$$\mathrm{T}(n) = \begin{cases} O(1) & \text{if } n = 1 \\ \underbrace{T(n-1)}_{\text{sorting both halves}} + \underbrace{O(n)}_{\text{dividing + merging}} & \text{otherwise} \end{cases}$$

Quick Sort: Best-case Running Time

The recurrence for the best-case running time T(n) is
[Partition is always balanced]

$$T(n) \le \begin{cases} O(1) & \text{if } n = 1 \\ \underbrace{T(\lceil n/2 \rceil)}_{\text{solve left half}} + \underbrace{T(\lfloor n/2 \rfloor)}_{\text{solve right half}} + \underbrace{O(n)}_{\text{dividing}} + \underbrace{0}_{\text{merging}} & \text{otherwise} \end{cases}$$

equivalently

$$T(n) = \begin{cases} O(1) & \text{if } n = 1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{O(n)}_{\text{dividing + merging}} & \text{otherwise} \end{cases}$$

Quick Sort: Running Time

- **In the worst case:**

$$T(n) = \begin{cases} b & \text{if } n = 1 \\ T(n-1) + bn & \text{if } n > 1 \end{cases}$$

By solving the recurrence, we get

$$T(n) = O(n^2)$$

- **In the best case:**

$$T(n) = \begin{cases} b & \text{if } n = 1 \\ 2T(n/2) + bn & \text{if } n > 1 \end{cases}$$

By solving the recurrence, we get

$$T(n) = O(n \log n)$$

Quick Sort: Analysis

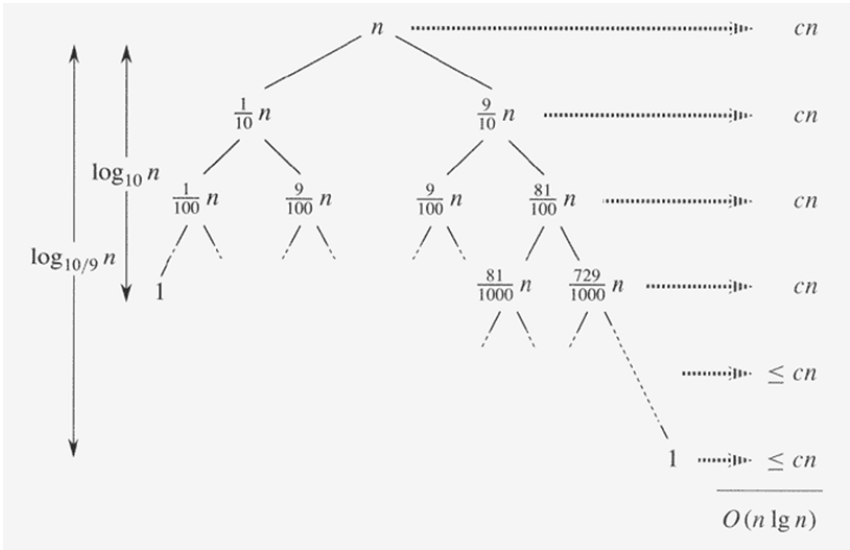
- The real liability of quicksort is that it runs in $O(n^2)$ on already-sorted input
- Book discusses two solutions:
  - Randomize the input array, OR
  - *Pick a random pivot element*
- *How will these solve the problem?*
  - By ensuring that no particular input can be chosen to make quicksort run in $O(n^2)$ time

Analyzing Quicksort: Average Case

- Assuming random input, average-case running time is much closer to $O(n \log n)$ than $O(n^2)$
- First, a more intuitive explanation/example:
  - Suppose that partition() always produces a 9-to-1 split. This looks quite unbalanced!
  - The recurrence is thus:

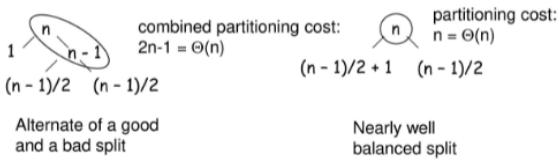
    $T(n) = T(9n/10) + T(n/10) + O(n)$
  - *How deep will the recursion go?* (draw it)

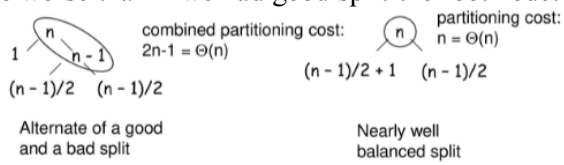
Analyzing Quicksort: Average Case



Analyzing Quicksort: Average Case

- Intuitively, a real-life run of quicksort will produce a mix of "bad" and "good" splits
  - Randomly distributed among the recursion tree
  - Pretend for intuition that they alternate between best-case ($n/2 : n/2$) and worst-case ($n - 1 : 1$)
  - *What happens if we bad-split root node, then good-split the resulting size ($n - 1$) node?*



Analyzing Quicksort: Average Case

- Intuitively, a real-life run of quicksort will produce a mix of "bad" and "good" splits
  - Randomly distributed among the recursion tree
  - Pretend for intuition that they alternate between best-case ($n/2 : n/2$) and worst-case ($n - 1 : 1$)
  - *What happens if we bad-split root node, then good-split the resulting size ($n - 1$) node?*
    - We end up with three subarrays, size 1, $(n-1)/2$, $(n-1)/2$
    - Combined cost of splits = $n + n - 1 = 2n - 1 = O(n)$
    - No worse than if we had good-split the root node!